Who Can Do What

The permission matrix, exactly as shipped. Written for whoever asks — a new administrator deciding which role to grant, or an auditor asking how separation of duties is enforced.

> This is the seeded matrix. It is editable without a software release, so if your deployment has been changed, this page describes the starting point rather than your current state. Your administrator can confirm.

The seven roles in one line each

ROLE	IN ONE LINE	TWO-FACTOR
Administrator	Everything, including anything a future release adds	**Required**
Finance	Invoicing, receipts, duty payment. Nothing operational.	**Required**
Operations Manager	Everything a coordinator does, plus the reversals, rate cards and reference data	No
Coordinator	Moves shipments forward. Cannot reverse them.	No
Sales	Enquiry to award. Nothing operational, nothing financial.	No
Customs Broker	Declarations, and the shipment fields clearance depends on	No
Read Only	Sees shipments and reports. Changes nothing.	No

Administrator is not a listed set of permissions. It receives every key, including keys added by future releases — so a new capability cannot ship unreachable. That is why it is not a role to hand out freely.

The full matrix

● held · ○ not held

PERMISSION	WHAT IT ALLOWS	ADMIN	FIN	OPS MGR	COORD	SALES	CUSTOMS	READ
`enquiry.assign`	Give an enquiry an owner	●	○	●	●	●	○	○
`enquiry.rate`	Price an enquiry	●	○	●	●	●	○	○
`enquiry.close`	Close won or lost	●	○	●	○	●	○	○
`enquiry.cancel`	Cancel an enquiry	●	○	●	○	●	○	○
`quote.send`	Send a quotation	●	○	●	●	●	○	○

PERMISSION	WHAT IT ALLOWS	ADMIN	FIN	OPS MGR	COORD	SALES	CUSTOMS	READ
`quote.revise`	Supersede with a new version	●	○	●	●	●	○	○
`quote.accept`	Record acceptance	●	○	●	○	●	○	○
`quote.reject`	Record rejection	●	○	●	○	●	○	○
`job.read`	See shipments	●	●	●	●	●	●	●
`job.book`	Move a shipment out of draft	●	○	●	●	○	○	○
`job.update`	Milestones, dates, references	●	○	●	●	○	●	○
`job.hold`	Place on hold and release	●	○	●	●	○	○	○
`job.complete`	Mark operationally finished	●	○	●	○	○	○	○
`job.close`	Close financially	●	○	●	○	○	○	○
`job.cancel`	Cancel a shipment	●	○	●	○	○	○	○
`customs.file`	Record a declaration	●	○	●	●	○	●	○
`customs.update`	Move it through its stages	●	○	●	●	○	●	○
`customs.pay`	Record duty paid	●	●	●	○	○	●	○
`document.generate`	Issue a generated document — house bill, delivery order, packing list, customs pack	●	○	●	●	○	○	○
`document.release`	Release a document to the customer	●	○	●	●	○	○	○
`consol.write`	Build a consolidation and attach houses	●	○	●	●	○	○	○
`consol.allocate`	Split the master cost across the houses	●	●	●	○	○	○	○
`invoice.approve`	Mark reviewed	●	●	○	○	○	○	○
`invoice.issue`	**Allocate the number. Irreversible.**	●	●	○	○	○	○	○

PERMISSION	WHAT IT ALLOWS	ADMIN	FIN	OPS MGR	COORD	SALES	CUSTOMS	READ
`invoice.void`	Cancel; the number stays used	●	●	○	○	○	○	○
`invoice.credit`	Raise a credit note	●	●	○	○	○	○	○
`payment.record`	Record and allocate a receipt	●	●	○	○	○	○	○
`finance.export`	Export financial data	●	●	○	○	○	○	○
`credit.override`	Let a shipment book past a credit hold, with a reason	●	●	●	○	○	○	○
`payable.record`	Capture a supplier bill	●	●	●	●	○	○	○
`payable.approve`	Approve a matched bill for payment	●	●	○	○	○	○	○
`payable.pay`	Record a supplier payment	●	●	○	○	○	○	○
`settlement.run`	Run and agree a partner settlement	●	●	●	○	○	○	○
`rate.card.write`	Create or change a rate card	●	○	●	○	○	○	○
`customer.write`	Create or edit customers	●	○	●	●	●	○	○
`partner.write`	Agents, hauliers, carriers	●	○	●	○	○	○	○
`report.read`	Dashboards and exports	●	●	●	○	●	○	●
`admin.reference.write`	Ports, charge types, milestones, calendars	●	○	●	○	○	○	○
`admin.users.write`	Users, roles, **and vendor access**	●	○	○	○	○	○	○
`admin.branches.write`	Branch configuration	●	○	○	○	○	○	○
`admin.audit.read`	The audit log	●	○	○	○	○	○	○
`admin.brand.read`	Branding tokens	●	○	○	○	○	○	○

PERMISSION	WHAT IT ALLOWS	ADMIN	FIN	OPS MGR	COORD	SALES	CUSTOMS	READ
`admin.api_keys.write`	API keys for the read API and webhooks	●	○	○	○	○	○	○
`admin.integrations.write`	Storage, mail, FX, carrier, model and e-invoice settings	●	○	○	○	○	○	○
`admin.mailboxes.write`	The inbound mailboxes, and their passwords	●	○	○	○	○	○	○

The four separations that are deliberate

These are the ones to be able to explain, because they are the ones that get asked about.

Finance alone may issue, void or credit an invoice

No operational role holds `invoice.issue`. An operations manager who could issue invoices is the classic segregation-of-duties failure, and it is the first thing an auditor tests.

The reverse holds too: **Finance cannot book, update or close a shipment.** If an invoice is wrong because the job is wrong, operations fixes the job and finance credits the invoice. Neither corrects the other's record.

A coordinator moves work forward; only a manager reverses it

`job.cancel`, `job.close` and `job.complete` sit with the operations manager. `enquiry.close` — deciding won or lost — sits with the manager and with sales.

Reversal is an escalation. It should cost a conversation, and the conversation is where the reason gets recorded.

Reference data is operational, not administrative

`admin.reference.write` is held by the operations manager, despite the `admin.` prefix. Adding a port when a new lane opens is daily work — and if it waits for an administrator, what actually happens is that somebody types the port into a notes field, and the system grows a free-text "Other" that swallows the data it was built to structure.

Vendor access is gated on `admin.users.write`

Granting an outsider access to the deployment is the same authority as granting a colleague access to a branch. Inventing a separate key would let someone hold one without the other, which is a distinction with no meaning.

Two things this matrix is not

It is not the security boundary. It decides what the interface *offers*. The real boundary is in the database: branch separation, immutability and audit are enforced by 197 policies across 82 tables, plus the state machines that reject an invalid transition regardless of who asked.

The asymmetry is deliberate. A permission check that is forgotten hides a button. A database policy that is forgotten leaks a branch. That is why the expensive guarantees live below the application.

It is not per-record. Permissions are per role; visibility is per branch. Holding `job.update` does not let you update every shipment — only those in branches where you hold a role. A request for a shipment in another branch returns **not found**, because the row is not visible to the query at all.

Roles are granted per branch

The same person can be a coordinator in Kuwait and read-only in Dubai. Grants are per branch, and there is no all-branches switch.

Nobody can widen their own access. Only an administrator can grant a role in another branch, and every grant is in the audit log.

Questions an auditor asks, and the answers

QUESTION	ANSWER
Can an operations manager issue an invoice?	No. `invoice.issue` is held by Finance and Administrator only.
Can an administrator alter the audit log?	No. Update and delete on it are revoked from every role in the database. The permission does not exist.
Can a user see another branch's data?	No. Enforced by database policy, not by the interface. The row is not visible to the query.
Can a user grant themselves access?	No. Only `admin.users.write` can, and every grant is recorded.
Can the supplier see our data?	Not by default and not without a grant from your administrator — scoped to branches, read or write, time-limited, against a ticket, revocable, and permanently recorded.
Is anything ever deleted?	No. Records are superseded, closed, cancelled or deactivated. Delete is revoked from the application's own account on every table.
Who can change the permission matrix itself?	An administrator, through the seed. It is editable without a software release, and the change is audited.